

**LAPORAN PRAKTIKUM
SISTEM OPERASI**

TUGAS 4

Dibuat untuk Memenuhi Tugas Mata Kuliah Sistem Operasi



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.K

Disusun Oleh:

Muhamad Gufron (2410651053)

**PROGRAM STUDI TEKNIK INFORMATIKA FAKULTAS
TEKNIK**

**UNIVERSITAS MUHAMMADIYAH JEMBER TAHUN
AKADEMIK 2024/2025**

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan tugas makalah yang berjudul “Memori Dalam Komputer” ini tepat waktu.

Adapun tujuan dari penulisan makalah ini adalah untuk memenuhi tugas dosen pada mata kuliah Dasar-Dasar Sistem. Selain itu, makalah ini juga bertujuan untuk menambah wawasan tentang Kerja Sama Kelompok bagi para pembaca dan juga bagi penulis.

Kami mengucapkan terima kasih kepada Triawan Adi Cahyanto, M.K selaku dosen mata kuliah Pengembangan Diri yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan bidang studi yang kami tekuni.

Kami juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga kami dapat menyelesaikan makalah ini

Kami menyadari, makalah yang kami tulis ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun akan kami nantikan demi kesempurnaan makalah ini.

PEMBAHASAN

BAB I

1.1. Eksperimen dengan Data Berbeda

Data Set 1

```
gufro@MSI:~$ cd ~/Praktikum4
gufro@MSI:~/Praktikum4$ nano compare.c
gufro@MSI:~/Praktikum4$ gcc compare.c -o compare
gufro@MSI:~/Praktikum4$ ./compare
```

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:

1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 3
Masukkan time quantum untuk Round Robin: 2

== Proses P1 ==
Arrival Time: 0
Burst Time: 10

== Proses P2 ==
Arrival Time: 1
Burst Time: 5

== Proses P3 ==
Arrival Time: 2
Burst Time: 8

HASIL ALGORITMA FCFS

PID	AT	BT	CT	TAT	WT
P1	0	10	10	10	0
P2	1	5	15	14	9
P3	2	8	23	21	13

Average Turnaround Time: 15.00

Average Waiting Time: 7.33

- Average TAT = $(10 + 14 + 21) / 3 = 45 / 3 = 15.00$

- $\text{Average WT} = (0 + 9 + 13) / 3 = 22 / 3 = 7.33$

berikut proses penjelasan pada FCFS :

- Pada proses FCFS dieksekusi berdasarkan urutan kedatangan Arrival time tanpa interupsi
- Karena p1 datang lebih dulu ($AT = 0$), CPU langsung menjalankan p1 hingga selesai.
- Setelah proses berikutnya p2 dan p3 di jalankan sesuai urutan kedatangannya
-

Perbedaan

Berikut perbedaan pada FCFS:

- Pada FCFS tidak bisa dihentikan (non-preemptive)
- Proses yang datang lebih awal mendapatkan prioritas penuh
- Proses yang datang belakangan P2 P3 harus menunggu lama jika proses pertama p1 memiliki waktu yang panjang.
- Menghasilkan waktu tunggu karena efek antrian panjang
- Pada algoritma sederhana ini mudah diimplementasikan, tapi tidak efisien jika tidak perbedaan burst time yang besar antar proses.

1.2. SJF

=====					
HASIL ALGORITMA SJF					
=====					
PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	10	10	10	0
P2	1	5	15	14	9
P3	2	8	23	21	13
Average Turnaround Time: 15.00					
Average Waiting Time: 7.33					

- Average TAT = 15.00
- Average WT = 7.33

Berikut proses penjelasan pada JSF:

- Pada SJF CPU akan memilih proses dengan burst time paling pendek dari proses yang sudah tiba di antrian
- Karena p1 datang pertama, maka CPU mulai dari p1
- Setelah p1 selesai, proses berikutnya yang tersedia adalah P2 (BT=5) dan P3 (BT=8) → CPU memilih **P2** karena BT lebih pendek.
- Setelah p2 selesai, CPU menjalankan P3 sampai terakhir.

Berikut perbedaan pada SJF:

- pada SJF memilih proses dengan burst time terpendek, tetapi pada data set 1 hasilnya sama seperti FCFS karena proses datang berurutan dan p1 langsung dijalankan sejak awal.
- Memberikan waktu tunggu dan turnaround time lebih efisien pada umumnya, jika banyak variasi burst time.
- Tidak adil bagi proses panjang, karena proses pendek selalu didahulukan.
- Cocok untuk sistem batch processing dimana lama proses bisa diperkirakan.
- Tidak cocok untuk sistem real time karena sulit memprediksi burst time.

1.3. Round Robin

```
=====
HASIL ALGORITMA Round Robin (Q=2)
=====
```

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	10	23	23	13
P2	1	5	15	14	9
P3	2	8	21	19	11

Average Turnaround Time: 18.67
Average Waiting Time: 11.00

- Average TAT = 18.67
- Average WT = 11.00

- Context Switch = 9

Berikut proses penjelasan pada Round Robin:

- Setiap proses mendapat jatah waktu eksekusi yang sama (quantum).
- Jika proses belum selesai dalam jatah waktunya, CPU akan menghentikan proses sementara (preemptive) dan memberi giliran ke proses berikutnya.
- Setelah semua proses mendapat giliran, proses yang belum selesai akan kembali ke antrian untuk giliran berikutnya.

Berikut perbedaan pada Round Robin :

- Bersifat preemptive, karena setiap proses hanya berjalan dalam batas quantum (2 unit waktu).
- Semua proses mendapatkan kesempatan secara bergiliran, sehingga lebih baik adil dibanding kan FCFS dan SJF.
- Namun, sering terjadi context switching, waktu total penyelesaian bisa menjadi lama
- Dan cocok untuk sistem multi tasking atau time-sharing.
- Kelemahannya pada turnaround time dan waiting time lebih besar dibandingkan SJF dan FCS pada data

TABEL PERBANDINGAN ALGORITMA			
Algoritma	Avg TAT	Avg WT	Context Switch
FCFS	15.00	7.33	2
SJF	15.00	7.33	2
Round Robin (Q=2)	18.67	11.00	9

KESIMPULAN:
 ✓ Algoritma terbaik (Avg TAT terkecil): FCFS (15.00)
 ✓ Algoritma terbaik (Avg WT terkecil): FCFS (7.33)

GRAFIK PERBANDINGAN AVERAGE WAITING TIME			
FCFS	7.33		
SJF	7.33		
Round Robin (Q=2)	11.00		

PROGRAM SELESAI

2.1. Eksperimen dengan Data Berbeda

Data Set 2

```
gufro@MSI:~/Praktikum4$ ./compare
```

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:

1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 4

Masukkan time quantum untuk Round Robin: 2

== Proses P1 ==

Arrival Time: 0

Burst Time: 5

== Proses P2 ==

Arrival Time: 1

Burst Time: 3

== Proses P3 ==

Arrival Time: 2

Burst Time: 8

== Proses P4 ==

Arrival Time: 3

Burst Time: 6

```
=====
HASIL ALGORITMA FCFS
=====
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0        5        5        5        0
P2        1        3        8        7        4
P3        2        8       16       14        6
P4        3        6       22       19       13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75
```


$$\text{Average Turnaround Time (TAT)} = (5 + 7 + 14 + 19) / 4 = 11.25$$

$$\text{Average Waiting Time (WT)} = (0 + 4 + 6 + 13) / 4 = 5.75$$

Berikut proses penjelasan pada FCFS:

- Proses dieksekusi berdasarkan urutan kedatangan (Arrival time)
- Karena p1 datang yang pertama kali maka AT=0 menjalankan p1 terlebih dahulu sehingga proses selesai.
- Setelah p1 selesai lanjut proses berikutnya (P2, P3, P4) dan di jalankan sesuai urutan kedatangan terlebih dahulu.

Berikut perbedaan pada FCFS:

- Pada proses FCFS tidak bisa dihentikan (non preemptive)
- Proses yang datang lebih awal maka mendapatkan prioritas penuh.
- Proses yang datang belakangan harus menunggu lama jika proses di awal memiliki burst time.

2.2. SJF

=====					
HASIL ALGORITMA SJF					
=====					
PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	22	20	12
P4	3	6	14	11	5
Average Turnaround Time: 10.75					
Average Waiting Time: 5.25					

$$\text{Average TAT} = 10.75$$

$$\text{Average WT} = 5.25$$

Berikut proses penjelasan pada SJS:

- Proses CPU memilih proses yang memiliki waktu eksekusi (BT) paling pendek dari proses yang sudah datang

- Setelah proses p1 selesai (BT=5) proses yang paling pendek di antaranya yang sudah tiba (P2=3, P3=8, P4=6) adalah p2, sehingga p2 dijalankan.
- Setelah P2 selesai, CPU memilih antara P3 dan P4 yang lebih pendek adalah P4 (BT=6), baru kemudian P3 (BT=8).

Berikut proses perbedaan pada SJS :

- Pada proses memilih proses dengan burt time paling pendek diantara yang siap.
- Memberikan waktu tunggu dan turnaround rata-rata terkecil dibanding FCFS dan RR.
- Dan tidak terhadap proses panjang p3 menunggu paling lama.
- Tisak cocok untuk sistem interaktif karena sulit memperkirakan waktu yang di eksekusi

2.3.Round Robin

```
=====
HASIL ALGORITMA Round Robin (Q=2)
=====
```

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	5	16	16	11
P2	1	3	11	10	7
P3	2	8	22	20	12
P4	3	6	20	17	11

Average Turnaround Time: 15.75
Average Waiting Time: 10.25

Tekan Enter untuk melihat perbandingan...

Average TAT = 15.75

Average WT = 10.25

Berikut proses penjelasan Round Robin:

- Setiap proses mendapat jatah waktu eksekusi tertentu (misalnya quantum = 2).
- Proses dijalankan bergiliran, dan jika belum selesai setelah jatah waktu habis, proses akan dimasukkan ke akhir antrian.
- Dengan data set ini, semua proses akan mendapat giliran secara bergantian.

Berikut perbedaan pada Round Robin:

- Bersifat preemptive setiap proses bisa dihentikan dan dilanjut nanti.
- Semua proses mendapat kesempatan sama – sama untuk dieksekusi dengan adil.
- Cocok untuk sistem interaktif (seperti OS multitasking).
- Waktu total eksekusi lebih lama karena sering terjadi context switch.

```
=====
                        TABEL PERBANDINGAN ALGORITMA
=====
```

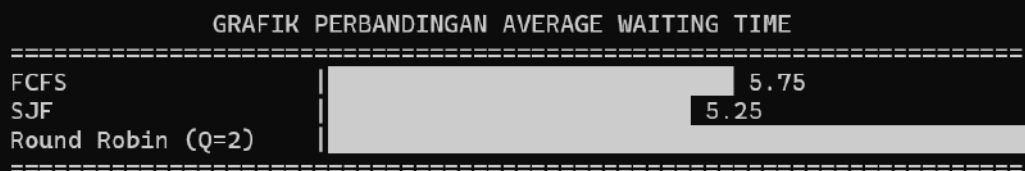
Algoritma	Avg TAT	Avg WT	Context Switch
FCFS	11.25	5.75	3
SJF	10.75	5.25	3
Round Robin (Q=2)	15.75	10.25	8

```
=====
```

KESIMPULAN:

✓ Algoritma terbaik (Avg TAT terkecil): SJF (10.75)

✓ Algoritma terbaik (Avg WT terkecil): SJF (5.25)



PROGRAM SELESAI

3.1. Analisis Eksperimen ketiga algoritma

1. Algoritma mana yang memberikan Average Waiting Time terkecil?

Karena dari hasil tabel percobaan, nilai Average waiting Time SJF paling kecil

Dibandingkan dengan FCFS dan Round Robin

Terjadi karena SJF Selalu menjalankan proses dengan waktu yang di eksekusi paling pendek terlebih dahulu tidak yang paling tinggi, sehingga total waktu yang ditunggu rata-rata menjadi lebih efisien.

2. Mengapa hasil algoritma berbeda-beda?

Karena setiap algoritma mempunyai cara kerja yang berbeda atau aturan penjadwalan

- Pada FCFS berdasarkan urutan yang hadir terlebih dahulu, jadi kalau ada proses dengan waktu yang lama hadir/datang terlebih dahulu, maka proses yang lain akan menunggu lama.
- Pada SJF memilih proses dengan waktu eksekusi paling pendek, sehingga lebih efisien dalam waktu yang ditunggu.
- Pada Round Robin membagi waktu CPU menjadi potongan kecil quantum dan memberi giliran ke setiap proses. Membuat sistem lebih adil, tetapi waktu yang di tunggu lebih baik adil, tapi waktu tunggu bisa lebih besar karena sering bergantian

3. Kapan sebaiknya menggunakan algoritma FCFS?

- FCFS ini cocok untuk sistem batch processing yang tidak butuh interaksi langsung.
- Dipakai kalau sederhana dan mudah di terapkan lebih diutamakan dari pada efisiensi waktu menunggu.
- Seperti
Contohnya : sistem antrian bensin. mobil pertama datang di isi terlebih dulu Lanjut mobil kedua harus menunggu mobil pertama selesai pengisian bensin, walau mobil kedua Cuma ngisi bensin sedikit. Jadi jika mobil pertama ngisi full maka yang dibelakang semua ikut menunggu lama.

4. Kapan sebaiknya menggunakan algoritma SJF?

- Digunakan jika durasi proses dapat diperkirakan dengan baik
- Tujuannya memperoleh waktu tunggu dan turnaround terkecil
- Cocok untuk sistem non-interaktif atau batch system.
- Contohnya
Seperti di hp, waktu mengedit foto, efek ringan seperti potongan atau atur kecerahan bisa langsung di jalankan terlebih dahulu, sedangkan memakai efek berat seperti filter dikerjakan belakangan. Tujuannya supaya pengguna lain nggak menunggu lama. Dengan cara ini waktu yang di tunggu rata-rata bisa lebih kecil dan pekerjaan terasa lebih cepat selesai.

5. Kapan sebaiknya menggunakan algoritma Round Robin?

- Cocok untuk sistem interaktif atau time sharing, seperti multi-user.
- Berguna saat keadilan penting, karena semua proses mendapatkan jatah CPU bergiliran.
- Cocok juga kalau sistem ini perlu respon cepat dari setiap proses pengguna.
- Contoh multitasking (windows, linux).

3.2. Modifikasi Program

1. Input.txt (FCFS)

```
GNU nano 7.2 input.txt *
4
1 0 24
2 1 3
3 2 3
4 3 6
```

```
#include <stdio.h>
```

```
struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int start_time;
    int completion_time;
    int turnaround_time;
    int waiting_time;
};
```

```
int main() {
    int n;
    FILE *input = fopen("input.txt", "r");
    if (input == NULL) {
        printf("Gagal membuka file input.txt. Pastikan file ada di folder yang sama.\n");
        return 1;
    }
}
```

```
fscanf(input, "%d", &n);
struct Process proc[n];
```

```
for (int i = 0; i < n; i++) {
    proc[i].pid = i + 1;
```

```

    fscanf(input, "%d %d", &proc[i].arrival_time, &proc[i].burst_time);
}
fclose(input);

int current_time = 0;
for (int i = 0; i < n; i++) {
    if (current_time < proc[i].arrival_time)
        current_time = proc[i].arrival_time;

    proc[i].start_time = current_time;
    proc[i].completion_time = current_time + proc[i].burst_time;
    proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;
    proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;
    current_time = proc[i].completion_time;
}

float total_tat = 0, total_wt = 0;

FILE *output = fopen("hasil_fcfs.txt", "w");
if (output == NULL) {
    printf("Gagal membuka file hasil_fcfs.txt untuk menyimpan hasil.\n");
    return 1;
}

printf("\n=== HASIL PENJADWALAN FCFS ===\n");
printf("PID\tAT\tBT\tST\tCT\tTAT\tWT\n");
printf("---\t--\t--\t--\t--\t---\t--\n");

fprintf(output, "=== HASIL PENJADWALAN FCFS ===\n");
fprintf(output, "PID\tAT\tBT\tST\tCT\tTAT\tWT\n");
fprintf(output, "---\t--\t--\t--\t--\t---\t--\n");

for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        proc[i].pid, proc[i].arrival_time, proc[i].burst_time,
        proc[i].start_time, proc[i].completion_time,
        proc[i].turnaround_time, proc[i].waiting_time);

    fprintf(output, "P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        proc[i].pid, proc[i].arrival_time, proc[i].burst_time,
        proc[i].start_time, proc[i].completion_time,
        proc[i].turnaround_time, proc[i].waiting_time);
}

```

```

        total_tat += proc[i].turnaround_time;
        total_wt += proc[i].waiting_time;
    }

    float avg_tat = total_tat / n;
    float avg_wt = total_wt / n;

    printf("\nAverage Turnaround Time: %.2f\n", avg_tat);
    printf("Average Waiting Time: %.2f\n", avg_wt);
    fprintf(output, "\nAverage Turnaround Time: %.2f\n", avg_tat);
    fprintf(output, "Average Waiting Time: %.2f\n", avg_wt);

    // Gantt Chart Detail
    printf("\n=== GANTT CHART (Detail) ===\n");
    fprintf(output, "\n=== GANTT CHART (Detail) ===\n");

    for (int i = 0; i < n; i++) {
        printf(" P%d |", proc[i].pid);
        fprintf(output, " P%d |", proc[i].pid);
    }

    printf("\n");
    fprintf(output, "\n");
    printf("%d", proc[0].start_time);
    fprintf(output, "%d", proc[0].start_time);

    for (int i = 0; i < n; i++) {
        printf("\t%d", proc[i].completion_time);
        fprintf(output, "\t%d", proc[i].completion_time);
    }

    printf("\n\nKeterangan:\n");
    fprintf(output, "\n\nKeterangan:\n");
    for (int i = 0; i < n; i++) {
        printf("P%d: Start = %d, Finish = %d\n", proc[i].pid, proc[i].start_time,
proc[i].completion_time);
        fprintf(output, "P%d: Start = %d, Finish = %d\n", proc[i].pid, proc[i].start_time,
proc[i].completion_time);
    }
    fclose(output);
    printf("\nHasil disimpan ke file hasil_fcfs.txt\n");
    return 0;
}

```

Berikut penjelasannya:

- Pada program ini untuk membuka file input.txt untuk membaca data proses.
- Pada CPU prosesnya dari 0
- Proses dieksekusi secara berurutan berdasarkan waktu datang.
- Jika CPU sedang mengeksekusi proses lain, proses harus menunggu dalam antrian

Proses penghitungan pada proses yang sudah selesai program:

- $CT(\text{Completion Time}) = \text{Start Time} + \text{Burst Time}$
- $TAT(\text{Turnaround Time}) = \text{Completion Time} - \text{Arrival Time}$
- $WT(\text{Waiting Time}) = \text{Turnaround Time} - \text{Burst Time}$

Maka hasil outpunya:

```
gufro@MSI:~/Praktikum4$ touch input.txt
gufro@MSI:~/Praktikum4$ nano input.txt
gufro@MSI:~/Praktikum4$ ./fcfs

=== HASIL PENJADWALAN FCFS ===
PID      AT      BT      ST      CT      TAT      WT
---      --      --      --      --      ---      --
P1        1        0        1        1        0        0
P2       24        2       24       26        2        0
P3        1        3       26       29       28       25
P4        3        2       29       31       28       26

Average Turnaround Time: 14.50
Average Waiting Time: 12.75

=== GANTT CHART (Detail) ===
| P1 | P2 | P3 | P4 |
1      1      26      29      31

Keterangan:
P1: Start = 1, Finish = 1
P2: Start = 24, Finish = 26
P3: Start = 26, Finish = 29
P4: Start = 29, Finish = 31

Hasil disimpan ke file hasil_fcfs.txt
```


2. Input2.txt(SJF)

```
GNU nano 7.2      input.txt *
4
1 0 24
2 1 3
3 2 3
4 3 6
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int arrival_time;
```

```
    int burst_time;
```

```
    int completion_time;
```

```
    int turnaround_time;
```

```
    int waiting_time;
```

```
    bool is_completed;
```

```
};
```

```
int main() {
```

```
    FILE *input = fopen("input2.txt", "r"); // <- baca input2.txt
```

```
    if (!input) {
```

```
        printf("Gagal membuka file input2.txt. Pastikan file ada di folder yang sama.\n");
```

```
        return 1;
```

```
    }
```

```

int n;

fscanf(input, "%d", &n);

struct Process proc[n];

for (int i = 0; i < n; i++) {

    fscanf(input, "%d %d %d", &proc[i].pid, &proc[i].arrival_time, &proc[i].burst_time);

    proc[i].is_completed = false;

    proc[i].completion_time = 0;

    proc[i].turnaround_time = 0;

    proc[i].waiting_time = 0;

}

fclose(input)

int current_time = 0;

int completed = 0;

int sequence[1000];

int seq_count = 0;


while (completed < n) {

    int shortest = -1;

    int min_burst = 1e9;

    for (int i = 0; i < n; i++) {

        if (!proc[i].is_completed && proc[i].arrival_time <= current_time) {

            if (proc[i].burst_time < min_burst) {

                min_burst = proc[i].burst_time;

                shortest = i;
            }
        }
    }

    // ... (rest of the scheduling logic)
}

```

```

    }

}

}

if (shortest == -1) {

    current_time++;

    continue;

}

sequence[seq_count++] = shortest;

current_time += proc[shortest].burst_time;

proc[shortest].completion_time = current_time;

proc[shortest].turnaround_time      =      proc[shortest].completion_time      -
proc[shortest].arrival_time;

proc[shortest].waiting_time         =      proc[shortest].turnaround_time         -
proc[shortest].burst_time;

proc[shortest].is_completed = true;

completed++;

}

```

```

FILE *output = fopen("hasil_sjf.txt", "w");

```

```

if (!output) {

    printf("Gagal membuka file hasil_sjf.txt untuk menulis.\n");

    return 1;

}

```

```

printf("\n=== HASIL PENJADWALAN SJF ===\n");

```

```

printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

printf("---\t--\t--\t--\t--\t--\n");

fprintf(output, "=== HASIL PENJADWALAN SJF ===\n");

fprintf(output, "PID\tAT\tBT\tCT\tTAT\tWT\n");

fprintf(output, "---\t--\t--\t--\t--\t--\n");

float total_tat = 0, total_wt = 0;

for (int i = 0; i < n; i++) {

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",

        proc[i].pid, proc[i].arrival_time, proc[i].burst_time,

        proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);

    fprintf(output, "P%d\t%d\t%d\t%d\t%d\t%d\n",

        proc[i].pid, proc[i].arrival_time, proc[i].burst_time,

        proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);

    total_tat += proc[i].turnaround_time;

    total_wt += proc[i].waiting_time;

}

printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);

printf("Average Waiting Time: %.2f\n", total_wt / n);

fprintf(output, "\nAverage Turnaround Time: %.2f\n", total_tat / n);

fprintf(output, "Average Waiting Time: %.2f\n", total_wt / n);

printf("\n=== GANTT CHART ===\n");

for (int i = 0; i < seq_count; i++) {

    printf(" P%d |", proc[sequence[i]].pid);

}

```

```

printf("\n0");

for (int i = 0; i < seq_count; i++) {

    printf("  %d", proc[sequence[i]].completion_time);

}

printf("\n");


fprintf(output, "\n=== GANTT CHART ===\n|");

for (int i = 0; i < seq_count; i++) fprintf(output, " P%d |", proc[sequence[i]].pid);

fprintf(output, "\n0");

for (int i = 0; i < seq_count; i++) fprintf(output, "          %d",
proc[sequence[i]].completion_time);

fprintf(output, "\n");


fclose(output);

printf("\nHasil juga disimpan ke file: hasil_sjf.txt\n");

return 0;

}

```

Berikut proses penjelasannya:

- Program ini akan membaca data dari file "input2.txt".
- Yang berisi informasi tentang jumlah proses secara detail masing-masing (PID, arrival time, burst time)
- Pada SJF mencari proses dengan burst time terpendek yang sudah datang (arrival time current time) dan belum selesai
- Jika proses sudah di temukan dan di eksekusi sampai selesai

Beikut hasil outputnya :

```

gufro@MSI:~/Praktikum4$ nano input.txt
gufro@MSI:~/Praktikum4$ touch input2.txt
gufro@MSI:~/Praktikum4$ nano input2.txt
gufro@MSI:~/Praktikum4$ ./sjf

=== HASIL PENJADWALAN SJF ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0      24      24      24        0
P2        1       3      27      26       23
P3        2       3      30      28       25
P4        3       6      36      33       27

Average Turnaround Time: 27.75
Average Waiting Time: 18.75

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
0   24  27  30  36

Hasil juga disimpan ke file: hasil_sjf.txt

```

3. Input1.txt (Round Robin)

```

GNU nano 7.2      input1.txt *
4

1 0 24
2 1 3
3 2 3
4 3 6

3

```

```

#include <stdio.h>

#include <stdbool.h>

struct Process {

    int pid;

    int arrival_time;

    int burst_time;

    int remaining_time;

    int completion_time;

    int turnaround_time;

    int waiting_time;

};

int main() {

    FILE *input = fopen("input1.txt", "r"); // <- baca input1.txt

    if (!input) {

        printf("Gagal membuka file input1.txt. Pastikan file ada di folder yang sama.\n");

        return 1;

    }

    int n, quantum;

    fscanf(input, "%d", &n);

    struct Process proc[n];

    for (int i = 0; i < n; i++) {

        fscanf(input, "%d %d %d", &proc[i].pid, &proc[i].arrival_time, &proc[i].burst_time);

        proc[i].remaining_time = proc[i].burst_time;

        proc[i].completion_time = 0;
    }
}

```

```

        proc[i].turnaround_time = 0;

        proc[i].waiting_time = 0;
    }

    fscanf(input, "%d", &quantum);

    fclose(input);

    int current_time = 0;

    int completed = 0;

    int queue[1000];

    int front = 0, rear = 0;

    bool in_queue[n];

    for (int i = 0; i < n; i++) in_queue[i] = false;

    int gantt_pid[5000];

    int gantt_start[5000];

    int gantt_end[5000];

    int gantt_count = 0;

    for (int i = 0; i < n; i++) {

        if (proc[i].arrival_time <= current_time && proc[i].remaining_time > 0 &&
!in_queue[i]) {

            queue[rear++] = i;

            in_queue[i] = true;

        }

    }

    while (completed < n) {

        if (front == rear) {

            current_time++;

```



```

    for (int i = 0; i < n; i++) {

        if (proc[i].arrival_time <= current_time && proc[i].remaining_time > 0 &&
!in_queue[i]) {

            queue[rear++] = i;

            in_queue[i] = true;

        }

    }

    continue;

}

int idx = queue[front++];

in_queue[idx] = false;

gantt_pid[gantt_count] = proc[idx].pid;

gantt_start[gantt_count] = current_time;

if (proc[idx].remaining_time > quantum) {

    current_time += quantum;

    proc[idx].remaining_time -= quantum;

    gantt_end[gantt_count] = current_time;

    gantt_count++;

} else {

    current_time += proc[idx].remaining_time;

    proc[idx].remaining_time = 0;

    proc[idx].completion_time = current_time;

    proc[idx].turnaround_time = proc[idx].completion_time - proc[idx].arrival_time;

    proc[idx].waiting_time = proc[idx].turnaround_time - proc[idx].burst_time;

    completed++;

```

```

        gantt_end[gantt_count] = current_time;

        gantt_count++;
    }

    for (int i = 0; i < n; i++) {

        if (proc[i].arrival_time <= current_time && proc[i].remaining_time > 0 &&
!in_queue[i]) {

            queue[rear++] = i;

            in_queue[i] = true;

        }

    }

    if (proc[idx].remaining_time > 0) {

        queue[rear++] = idx;

        in_queue[idx] = true;

    }

}

FILE *output = fopen("hasil_roundrobin.txt", "w");

if (!output) {

    printf("Gagal membuka file hasil_roundrobin.txt untuk menulis.\n");

    return 1;

}

printf("\n=== HASIL PENJADWALAN ROUND ROBIN ===\n");

printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

printf("---\t--\t--\t--\t---\t--\n");

fprintf(output, "=== HASIL PENJADWALAN ROUND ROBIN ===\n");

```

```

fprintf(output, "PID\tAT\tBT\tCT\tTAT\tWT\n");

fprintf(output, "---\t--\t--\t--\t--\t--\n");

float total_tat = 0, total_wt = 0;

for (int i = 0; i < n; i++) {

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",

        proc[i].pid, proc[i].arrival_time, proc[i].burst_time,

        proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);

    fprintf(output, "P%d\t%d\t%d\t%d\t%d\t%d\n",

        proc[i].pid, proc[i].arrival_time, proc[i].burst_time,

        proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);

    total_tat += proc[i].turnaround_time;

    total_wt += proc[i].waiting_time;

}

printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);

printf("Average Waiting Time: %.2f\n", total_wt / n);

fprintf(output, "\nAverage Turnaround Time: %.2f\n", total_tat / n);

fprintf(output, "Average Waiting Time: %.2f\n", total_wt / n);

printf("\n=== GANTT CHART (Detail) ===\n");

for (int i = 0; i < gantt_count; i++) {

    printf(" P%d |", gantt_pid[i]);

}

printf("\n");

for (int i = 0; i < gantt_count; i++) {

    if (i == 0) printf("%d", gantt_start[i]);

```

```

        printf("  %d", gantt_end[i]);
    }

    printf("\n");

    fprintf(output, "\n=== GANTT CHART (Detail) ===\n");

    for (int i = 0; i < gantt_count; i++) fprintf(output, " P%d |", gantt_pid[i]);

    fprintf(output, "\n");

    for (int i = 0; i < gantt_count; i++) {

        if (i == 0) fprintf(output, "%d", gantt_start[i]);

        fprintf(output, "  %d", gantt_end[i]);

    }

    fprintf(output, "\n");

    fclose(output);

    printf("\nHasil juga disimpan ke file: hasil_roundrobin.txt\n");

    return 0;

}

```

Berikut proses penjelasannya:

- Program ini akan membaca data dari file "input1.txt".
- Pada proses menggunakan quantum time (3unit) sebagai bentuk waktu maksimum pada proses yang di eksekusi selama quantum time sampai selesai jika ada sisa waktu
- Jika ada proses yang belum selesai maka dikembalikan pada proses antrian.

Berikut hasil outputnya

```
gufro@MSI:~/Praktikum4$ nano round_robin.c
gufro@MSI:~/Praktikum4$ gcc round_robin.c -o round_robin
gufro@MSI:~/Praktikum4$ touch input1.txt
gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ ./round_robin
```

=== HASIL PENJADWALAN ROUND ROBIN ===

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	24	0	0	0
P2	1	3	9	8	5
P3	2	3	12	10	7
P4	3	6	27	24	18

Average Turnaround Time: 10.50

Average Waiting Time: 7.50

=== GANTT CHART (Detail) ===

P1	P1	P2	P3	P4	P1	P1	P1	P4	P4
0 3	6	9	12	15	18	21	24	27	27

Hasil juga disimpan ke file: hasil_roundrobin.txt

```
gufro@MSI:~/Praktikum4$ |
```

3.2. Eksperimen dengan data berbeda untuk perbandingan algoritma

➤ FCFS

#Kode input.txt

```
GNU nano 7.2      input.txt *
4
1 0 24
2 1 3
3 2 3
4 3 6
```

hasil Outputnya akan menjadi seperti ini

```
gufro@MSI:~/Praktikum4$ touch input.txt
gufro@MSI:~/Praktikum4$ nano input.txt
gufro@MSI:~/Praktikum4$ ./fcfs

=== HASIL PENJADWALAN FCFS ===
PID      AT      BT      ST      CT      TAT      WT
---      --      --      --      --      ---      --
P1        1        0        1        1        0        0
P2       24        2       24       26        2        0
P3        1        3       26       29       28       25
P4        3        2       29       31       28       26

Average Turnaround Time: 14.50
Average Waiting Time: 12.75

=== GANTT CHART (Detail) ===
| P1 | P2 | P3 | P4 |
1      1      26      29      31

Keterangan:
P1: Start = 1, Finish = 1
P2: Start = 24, Finish = 26
P3: Start = 26, Finish = 29
P4: Start = 29, Finish = 31

Hasil disimpan ke file hasil_fcfs.txt
```

#Kode input.txt

```
GNU nano 7.2          input.txt *
5

1 0 10
2 0 5
3 0 8
4 0 12
5 0 6
```

hasil Outputnya akan menjadi seperti ini

```
gufro@MSI:~/Praktikum4$ nano input.txt
gufro@MSI:~/Praktikum4$ ./fcfs

=== HASIL PENJADWALAN FCFS ===
PID      AT      BT      ST      CT      TAT      WT
---      --      --      --      --      ---      --
P1        1        0        1        1        0        0
P2        8        2        8       10        2        0
P3        0        4       10       14       14       10
P4        3        0       14       14       11       11
P5        9        4       14       18        9        5

Average Turnaround Time: 7.20
Average Waiting Time: 5.20

=== GANTT CHART (Detail) ===
| P1 | P2 | P3 | P4 | P5 |
1      1      10      14      14      18

Keterangan:
P1: Start = 1, Finish = 1
P2: Start = 8, Finish = 10
P3: Start = 10, Finish = 14
P4: Start = 14, Finish = 14
P5: Start = 14, Finish = 18

Hasil disimpan ke file hasil_fcfs.txt
```

#Kode input.txt

```
GNU nano 7.2          input.txt *
5

1 0 8
2 2 4
3 4 9
4 6 5
5 8 3
```

hasil Outputnya akan menjadi seperti ini

```
gufro@MSI:~/Praktikum4$ nano input.txt
gufro@MSI:~/Praktikum4$ nano input.txt
gufro@MSI:~/Praktikum4$ ./fcfs

=== HASIL PENJADWALAN FCFS ===
PID      AT      BT      ST      CT      TAT      WT
---      --      --      --      --      ---      --
P1        1        0        1        1        0        0
P2        8        2        8       10        2        0
P3        2        4       10       14       12        8
P4        3        4       14       18       15       11
P5        9        4       18       22       13        9

Average Turnaround Time: 8.40
Average Waiting Time: 5.60

=== GANTT CHART (Detail) ===
| P1 | P2 | P3 | P4 | P5 |
1      1      10      14      18      22

Keterangan:
P1: Start = 1, Finish = 1
P2: Start = 8, Finish = 10
P3: Start = 10, Finish = 14
P4: Start = 14, Finish = 18
P5: Start = 18, Finish = 22

Hasil disimpan ke file hasil_fcfs.txt
```


#Kode input.txt

```
GNU nano 7.2          input2.txt *
4

1 0 24
2 1 3
3 2 3
4 3 6
```

hasil Outputnya akan menjadi seperti ini

```
gufro@MSI:~/Praktikum4$ nano input.txt
gufro@MSI:~/Praktikum4$ touch input2.txt
gufro@MSI:~/Praktikum4$ nano input2.txt
gufro@MSI:~/Praktikum4$ ./sjf

=== HASIL PENJADWALAN SJF ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0      24      24      24        0
P2        1       3      27      26       23
P3        2       3      30      28       25
P4        3       6      36      33       27

Average Turnaround Time: 27.75
Average Waiting Time: 18.75

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
0   24  27  30  36

Hasil juga disimpan ke file: hasil_sjf.txt
```

#Kode input2.txt

```
GNU nano 7.2      input2.txt *
5

1 0 10
2 0 5
3 0 8
4 0 12
5 0 6
```

hasil Outputnya akan menjadi seperti ini

```
gufro@MSI:~/Praktikum4$ nano input2.txt
gufro@MSI:~/Praktikum4$ ./sjf

=== HASIL PENJADWALAN SJF ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0       10      29      29       19
P2        0        5       5       5        0
P3        0        8      19      19       11
P4        0       12      41      41       29
P5        0        6      11      11        5

Average Turnaround Time: 21.00
Average Waiting Time: 12.80

=== GANTT CHART ===
| P2 | P5 | P3 | P1 | P4 |
0   5  11  19  29  41

Hasil juga disimpan ke file: hasil_sjf.txt
```

#Kode input2.txt

```
GNU nano 7.2          input2.txt *
5

1 0 8
2 2 4
3 4 9
4 6 5
5 8 3|
```

hasil Outputnya akan menjadi seperti ini

```
gufro@MSI:~/Praktikum4$ nano input2.txt
gufro@MSI:~/Praktikum4$ ./sjf

=== HASIL PENJADWALAN SJF ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0        8        8        8        0
P2        2        4       15       13        9
P3        4        9       29       25       16
P4        6        5       20       14        9
P5        8        3       11        3        0

Average Turnaround Time: 12.60
Average Waiting Time: 6.80

=== GANTT CHART ===
| P1 | P5 | P2 | P4 | P3 |
0   8  11  15  20  29

Hasil juga disimpan ke file: hasil_sjf.txt
```

#Kode input1.txt

```
GNU nano 7.2      input1.txt *
4
1 0 24
2 1 3
3 2 3
4 3 6
3
```

hasil Outputnya akan menjadi seperti ini

```
gufro@MSI:~/Praktikum4$ nano round_robin.c
gufro@MSI:~/Praktikum4$ gcc round_robin.c -o round_robin
gufro@MSI:~/Praktikum4$ touch input1.txt
gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ ./round_robin

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0      24      0        0        0
P2        1       3       9       8        5
P3        2       3      12      10       7
P4        3       6      27      24      18

Average Turnaround Time: 10.50
Average Waiting Time: 7.50

=== GANTT CHART (Detail) ===
| P1 | P1 | P2 | P3 | P4 | P1 | P1 | P1 | P4 | P4 |
0   3   6   9  12  15  18  21  24  27  27

Hasil juga disimpan ke file: hasil_roundrobin.txt
gufro@MSI:~/Praktikum4$ |
```

#Kode input1.txt

```
GNU nano 7.2          input1.txt *
5
1 0 10
2 0 5
3 0 8
4 0 12
5 0 6
4
```

hasil Outputnya akan menjadi seperti ini

```
gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ ./round_robin

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0       10      26      26      16
P2        0        5      27      27      22
P3        0        8      31      31      23
P4        0       12       0        0        0
P5        0        6       0        0        0

Average Turnaround Time: 16.80
Average Waiting Time: 12.20

=== GANTT CHART (Detail) ===
| P1 | P2 | P3 | P4 | P5 | P1 | P1 | P2 | P2 | P3 | P3 |
0   4   8  12  16  20  24  26  27  27  31  31

Hasil juga disimpan ke file: hasil_roundrobin.txt
```

#Kode input1.txt

```
GNU nano 7.2          input1.txt *
5

1 0 8
2 2 4
3 4 9
4 6 5
5 8 3

2
```

hasil Outputnya akan menjadi seperti ini

```
gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ ./round_robin

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0        8       16       16        8
P2        2        4       16       14       10
P3        4        9        0        0         0
P4        6        5        0        0         0
P5        8        3        0        0         0

Average Turnaround Time: 6.00
Average Waiting Time: 3.60

=== GANTT CHART (Detail) ===
| P1 | P1 | P2 | P1 | P1 | P3 | P1 | P2 | P4 | P2 | P1 |
0   2   4   6   8   10  12  12  14  16  16  16

Hasil juga disimpan ke file: hasil_roundrobin.txt
```

3.3. Analisis Eksperimen perbandingan algoritma

Setelah menjalankan program dengan berbagai test case, jawab pertanyaan berikut:

1. Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?
FCFS akan memberikan hasil terbaik ketika semua proses memiliki waktu eksekusi yang sama dan datangnya berurutan. contohnya seperti pengantrian pom bensin
- Mengapa SJF hampir selalu memberikan Average WT terkecil?
Karena pada SJF Menempatkan pekerjaan berdurasi pendek lebih awal mengurangi jumlah proses yang menunggu waktu lama, sehingga mengurangi total waktu tunggu yang dijumlahkan untuk semua proses
- Apa trade-off dari Round Robin?
Pada algoritma Round Robin terdapat antara kecepatan tanggapan sistem dan efisiensi eksekusi. Quantum yang kecil membuat proses mendapat giliran lebih cepat sehingga sistem terasa responsif, Sementara quantum besar memang mengurangi perpindahan konteks, namun membuat waktu tunggu proses lain meningkat dan sistem terasa lambat.

2. Context Switching

- Algoritma mana yang paling banyak melakukan context switch?
pada algoritma Round Robin menghasilkan jumlah context switch paling tinggi dibandingkan algoritma lain seperti FCFS dan SJF.
menyebabnya adalah karena setiap proses hanya dijalankan dalam jangka waktu singkat (sesuai nilai quantum) Semakin kecil nilai quantum, semakin sering pergantian proses terjadi.
- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin ?
Semakin kecil nilai quantum yang digunakan dalam algoritma Round Robin, maka semakin sering CPU melakukan perpindahan konteks antar proses
Jika quantum lebih besar maka context switch menjadi lebih sedikit, proses yang lain menunggu lama
- Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi ?

```

GNU nano 7.2                                input1.txt *
4
1 0 24
2 1 3
3 2 3
4 3 6
3

```

```

gufro@MSI:~/Praktikum4$ nano round_robin.c
gufro@MSI:~/Praktikum4$ gcc round_robin.c -o round_robin
gufro@MSI:~/Praktikum4$ touch input1.txt
gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ ./round_robin

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0      24      0        0        0
P2        1        3        9        8        5
P3        2        3       12       10        7
P4        3        6       27       24       18

Average Turnaround Time: 10.50
Average Waiting Time: 7.50

=== GANTT CHART (Detail) ===
| P1 | P1 | P2 | P3 | P4 | P1 | P1 | P1 | P4 | P4 |
0   3   6   9  12  15  18  21  24  27  27

Hasil juga disimpan ke file: hasil_roundrobin.txt
gufro@MSI:~/Praktikum4$ |

```

➤ 2

```

GNU nano 7.2                                input1.txt
3
1 5
2 3
3 1
2

```

```

gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ ./round_robin

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        1        5        9        8        3
P2        2        3        0        0        0
P3        3        1        8        5        4

Average Turnaround Time: 4.33
Average Waiting Time: 2.33

```


➤ 4

```
GNU nano 7.2                                     input1.txt
3
1 5
2 3
3 1
4
```

```
gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ ./round_robin

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        1        5        6        5        0
P2        2        3        9        7        4
P3        3        1       10        7        6

Average Turnaround Time: 6.33
Average Waiting Time: 3.33

=== GANTT CHART ===
| P1 | P1 | P2 | P3 |
0   5   6   9  10
```

➤ 8

```
GNU nano 7.2                                     input1.txt *
3
1 5
2 3
3 1
8
```

```
gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ ./round_robin

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        1        5        6        5        0
P2        2        3        9        7        4
P3        3        1       10        7        6

Average Turnaround Time: 6.33
Average Waiting Time: 3.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0   6   9  10
```

```

GNU nano 7.2                                input1.txt
3
1 5
2 3
3 1
16

gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ nano input1.txt
gufro@MSI:~/Praktikum4$ ./round_robin

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        1        5        6        5        0
P2        2        3        9        7        4
P3        3        1       10        7        6

Average Turnaround Time: 6.33
Average Waiting Time: 3.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0   6   9  10

```

Pada proses nilai quantum yang terlalu kecil menyebabkan sistem menjadi responsif namun tidak efisien karena banyak terjadi context switch. sedangkan, quantum yang terlalu besar membuat sistem lebih efisien tetapi respon proses menjadi lambat. Oleh karena itu, pemilihan quantum yang seimbang seperti 4 atau 8 memberikan hasil yang optimal.

3. Fairness

- Algoritma mana yang paling "adil" untuk semua proses?
Round Robin umumnya paling adil karena setiap proses mendapat giliran CPU sehingga tidak ada proses yang dibiarkan menunggu terus-meneru
- Proses mana yang paling dirugikan di algoritma FCFS?
proses yang paling dirugikan adalah proses yang datang belakangan dengan waktu eksekusi yang paling pendek, karena proses ini harus menunggu proses yang Panjang yang sudah datang duluan
- Apakah ada kemungkinan starvation di SJF?
Pada algoritma SJF terjadi jika selalu ada proses baru yang lebih pendek. Proses yang memiliki waktu eksekusi lebih lama akan terus tertunda karena selalu dikalahkan oleh proses yang lebih cepat.

4. Real World Application

- Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?

Dengan menggunakan Algoritma yang cocok untuk web server dengan banyak request kecil adalah SJF, karena dapat mempercepat respon dan meminimalkan waktu tunggu pengguna

- Untuk render video (task besar), algoritma mana yang cocok?

Algoritma yang cocok untuk render video adalah FCFS, karena efisien untuk proses besar dan tidak sering mengganti konteks

- Untuk OS desktop (interactive), algoritma mana yang cocok?

Untuk sistem operasi desktop Algoritma yang cocok untuk OS desktop adalah Round Robin, karena membuat sistem lebih responsif dan adil bagi proses interaktif